HW Portfolio

최 세 현

목차

교육/프로젝트

1. FPGA-Based Simple MIPS Processor

개인/프로젝트

1. 공정 미세화 한계 완화를 위한 PIM 개념의 Near-Memory 하드웨어 가속기 및 AXI 기반 SoC 통합 설계

교육/프로젝트

PROJECT 01. FPGA-Based Simple MIPS Processor

수행기간

2026.01

수행인원

1명

사용기술

VIVADO(verilog), FPGA

수행내용

(1) 기본적인 MIPS 명령어 셋을 처리할 수 있는 데이터패스 및 제어 유닛 설계

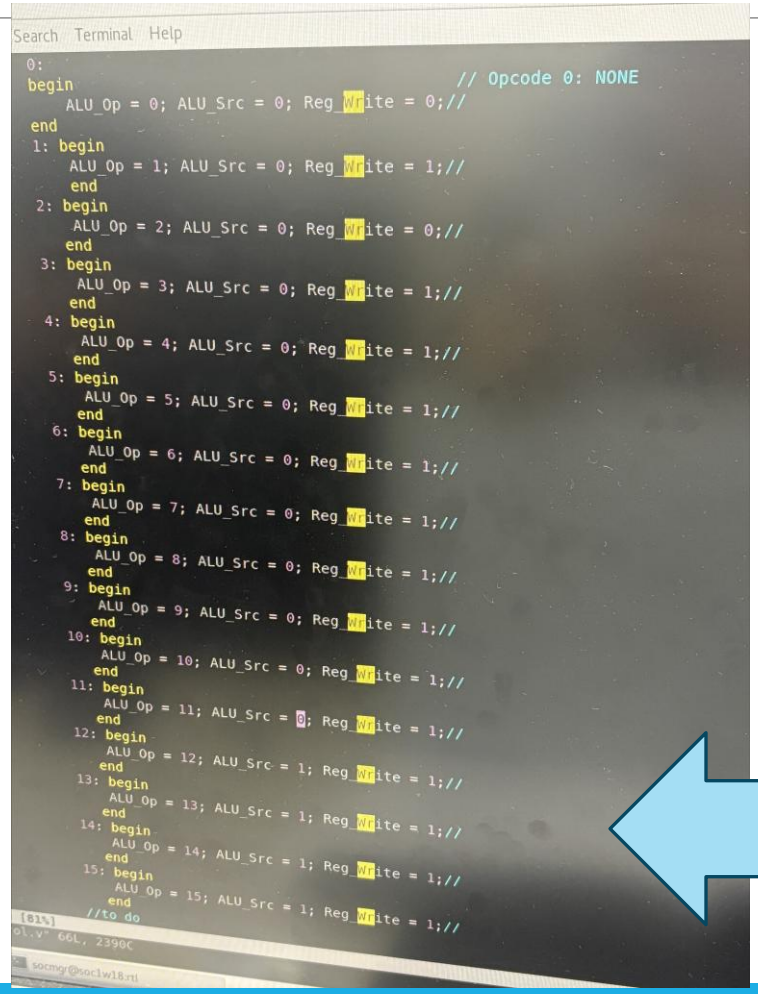
(2) ALU 내부에 CLA를 직접 설계 및 적용하여, 기존 RCA 대비 덧셈 연산 지연시간 개선

(3) UART 통신 프로토콜을 구현하여, PC로부터 직접 연산용 데이터를 입력받아 처리하는 I/O 환경 구축

(4) FSM 기반의 상태 제어 로직을 구현하고, FPGA 보드의 푸시 버튼을 활용한 사이클 단위 디버깅 및 동작 검증 완료

FPGA-Based Simple MIPS Processor

Control 모듈에서 각 opcode의 값에 따른 output값 설정

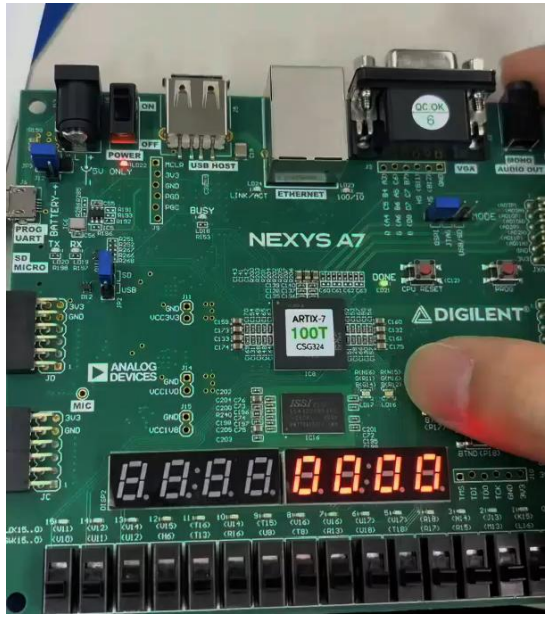


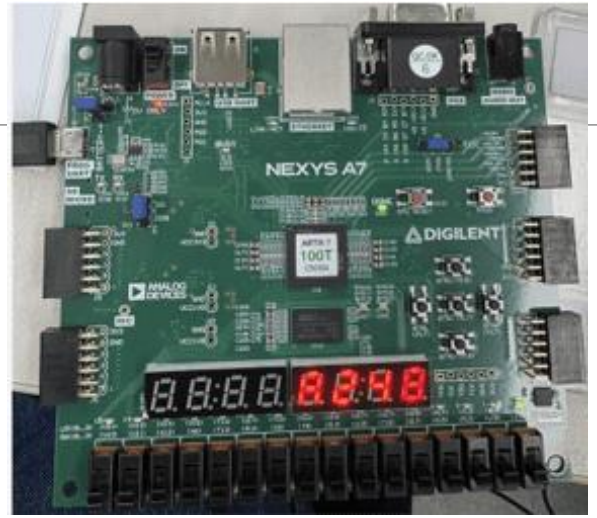
```
Search  Terminal  Help
0:                                          // Opcode 0: NONE
begin
    ALU_Op = 0; ALU_Src = 0; Reg_Write = 0;//
end
1: begin
    ALU_Op = 1; ALU_Src = 0; Reg_Write = 1;//
    end
2: begin
    ALU_Op = 2; ALU_Src = 0; Reg_Write = 0;//
    end
3: begin
    ALU_Op = 3; ALU_Src = 0; Reg_Write = 1;//
    end
4: begin
    ALU_Op = 4; ALU_Src = 0; Reg_Write = 1;//
    end
5: begin
    ALU_Op = 5; ALU_Src = 0; Reg_Write = 1;//
    end
6: begin
    ALU_Op = 6; ALU_Src = 0; Reg_Write = 1;//
    end
7: begin
    ALU_Op = 7; ALU_Src = 0; Reg_Write = 1;//
    end
8: begin
    ALU_Op = 8; ALU_Src = 0; Reg_Write = 1;//
    end
9: begin
    ALU_Op = 9; ALU_Src = 0; Reg_Write = 1;//
    end
10: begin
    ALU_Op = 10; ALU_Src = 0; Reg_Write = 1;//
    end
11: begin
    ALU_Op = 11; ALU_Src = 0; Reg_Write = 1;//
    end
12: begin
    ALU_Op = 12; ALU_Src = 1; Reg_Write = 1;//
    end
13: begin
    ALU_Op = 13; ALU_Src = 1; Reg_Write = 1;//
    end
14: begin
    ALU_Op = 14; ALU_Src = 1; Reg_Write = 1;//
    end
15: begin
    ALU_Op = 15; ALU_Src = 1; Reg_Write = 1;//
    end
//to do
[81%]
ol.v" 66L, 2390C
socmg@soc1w18:rtl
```

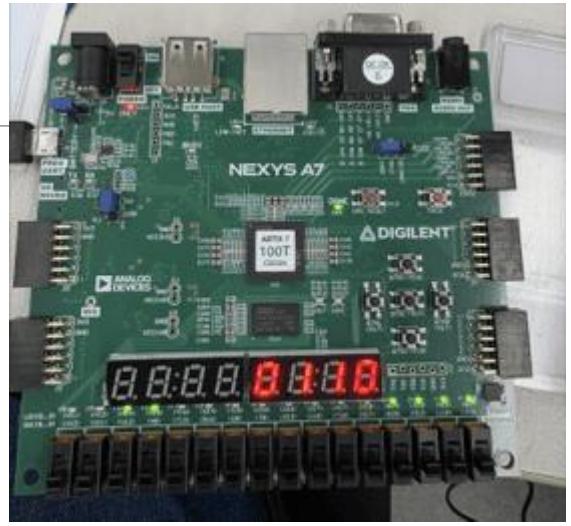
Opcode 12~15는 레지스터가 아닌 외부(pc)의 입력을 받아 사용하므로 ALU_Src = 1

Final Project Milestone1

- A248 → B84A
- 시뮬레이션의 결과를 FPGA에서 확인







Final Project Milestone2 - CLA : 병렬 carry 계산, 이전 carry직접 사용x

```
Applications  Places  Terminal                                socmgr
File  Edit  View  Search  Terminal  Help
1  `timescale 1ns / 1ps
2
3  module rca_4(
4      input       [ 3: 0] x,
5      input       [ 3: 0] y,
6      input               cin,
7      output              cout,
8      output      [ 3: 0] sum
9      );
10
11     wire [3:0] a,b;
12     wire [4:0] c;
13
14     assign c[0] = cin;
15     assign c[4] = cout;
16
17     assign a = x&y; //carry check
18     assign b = x^y;
19
20     assign sum[0] = c[0] ^ b[0];
21     assign sum[1] = c[1] ^ b[1];
22     assign sum[2] = c[2] ^ b[2];
23     assign sum[3] = c[3] ^ b[3]; // = x^y^c
24
25     assign c[1] = a[0] | (b[0] & c[0]); //a==1 : carry happen, a==0 but if agree? because of c
26     assign c[2] = a[1] | (b[1] & a[0]) | (b[1] & b[0] & c[0]); //if at [0], x=1,y=1? at least 1 hap
27     //and if at c[1] second condition, at least 1 (already carry at last stage
28     // = a[1] | (b[1] & c[1])
29     assign c[3] = a[2] | (b[2] & a[1]) | (b[2] & b[1] & a[0]) | (b[2] & b[1] & b[0] & c[0]);
30     assign c[4] = a[3] | (b[3] & a[2]) | (b[3] & b[2] & a[1]) | (b[3] & b[2] & b[1] & a[0]) |
31     (b[3] & b[2] & b[1] & b[0] & c[0]); // = cout
32 /*
33     fa_1    u0      (.a(x[0]), .b(y[0]), .cin(cin), .cout(c1),   .sum(sum[0]));
34     fa_1    u1      (.a(x[1]), .b(y[1]), .cin(c1),  .cout(c2),   .sum(sum[1]));
35     fa_1    u2      (.a(x[2]), .b(y[2]), .cin(c2),  .cout(c3),   .sum(sum[2]));
36     fa_1    u3      (.a(x[3]), .b(y[3]), .cin(c3),  .cout(cout), .sum(sum[3]));
37     */
38 endmodule
```

x	y	c	sum	Carry(out)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

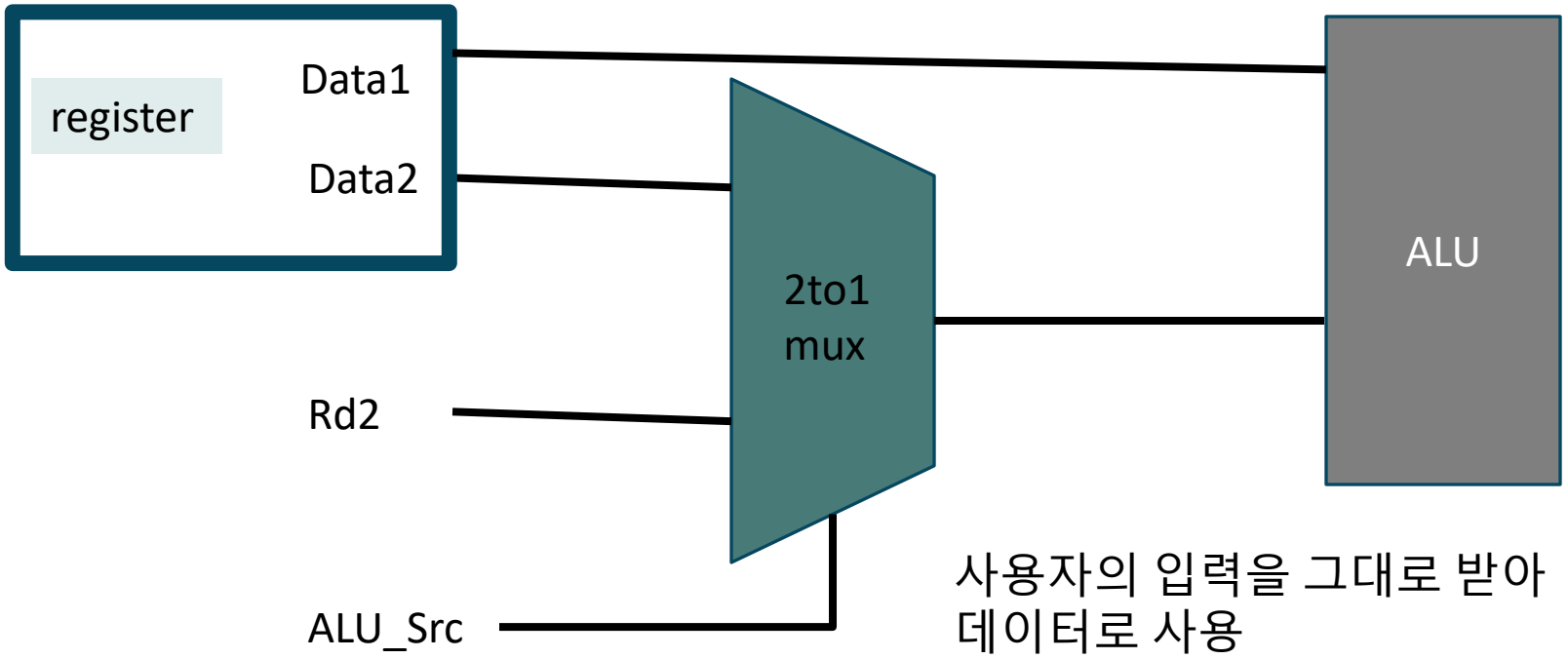
- Sum = x,y,c의 xor
- Carry out = xy+c(x^y)
- C[4] = 최종 cout
- 치환사용

Final Project Milestone3

socmgr@soc1w18:rtl

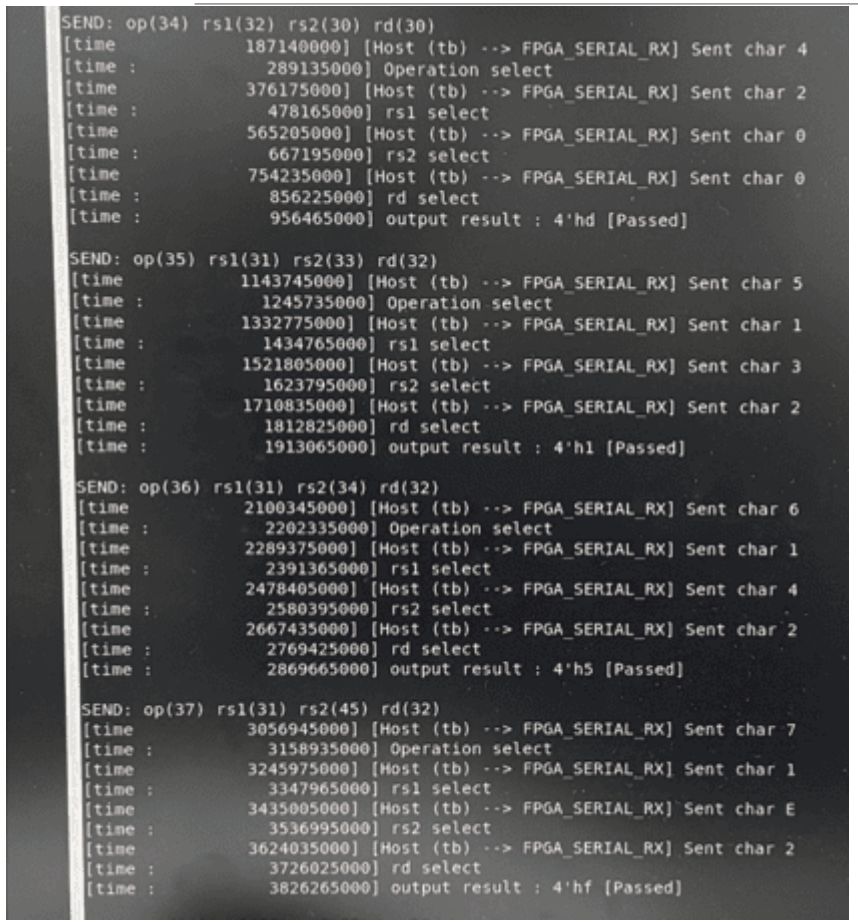
```
1  `timescale 1ns/1ps
2
3  module mux(a,b,sel,o);
4
5  input [3:0] a,b;
6  input sel;
7  output reg [3:0]o;
8
9  always @ (*) begin
10 if(sel==0)
11     o = a;
12 else
13     o = b;
14 end
15 endmodule
```

```
77  //todo
78  MUX mux0(.a(Data2),.b(Rd2),.sel(ALU_Src),.o(out));
79
80  //assign out =(ALU_Src == 1)?
81  //
```

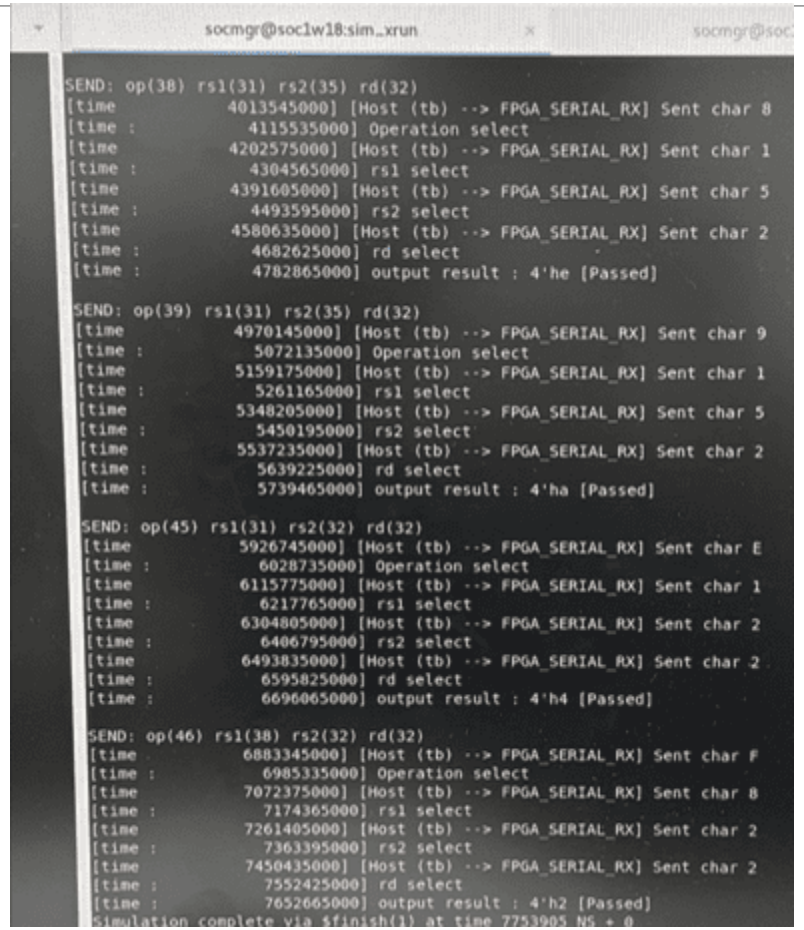


사용자의 입력을 그대로 받아
데이터로 사용

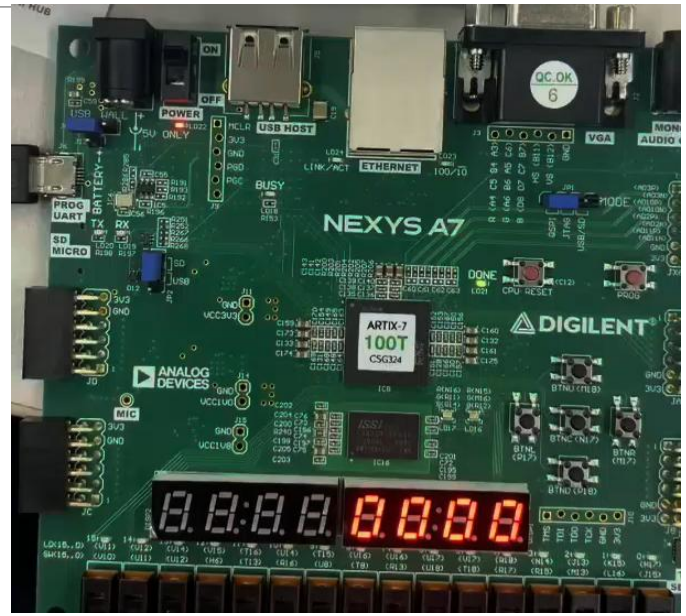
Final Project Milestone4



```
SEND: op(34) rs1(32) rs2(30) rd(30)
[time            187140000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 4
[time :            289135000] Operation select
[time            376175000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :            478165000] rs1 select
[time            565205000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 0
[time :            667195000] rs2 select
[time            754235000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 0
[time :            856225000] rd select
[time :            956465000] output result : 4'hd [Passed]

SEND: op(35) rs1(31) rs2(33) rd(32)
[time           1143745000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 5
[time :           1245735000] Operation select
[time           1332775000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           1434765000] rs1 select
[time           1521805000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 3
[time :           1623795000] rs2 select
[time           1710835000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           1812825000] rd select
[time :           1913065000] output result : 4'h1 [Passed]

SEND: op(36) rs1(31) rs2(34) rd(32)
[time           2100345000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 6
[time :           2202335000] Operation select
[time           2289375000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           2391365000] rs1 select
[time           2478405000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 4
[time :           2580395000] rs2 select
[time           2667435000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           2769425000] rd select
[time :           2869665000] output result : 4'h5 [Passed]

SEND: op(37) rs1(31) rs2(45) rd(32)
[time           3056945000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 7
[time :           3158935000] Operation select
[time           3245975000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           3347965000] rs1 select
[time           3435005000] [Host (tb) --> FPGA_SERIAL_RX] Sent char E
[time :           3536995000] rs2 select
[time           3624035000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           3726025000] rd select
[time :           3826265000] output result : 4'hf [Passed]
```

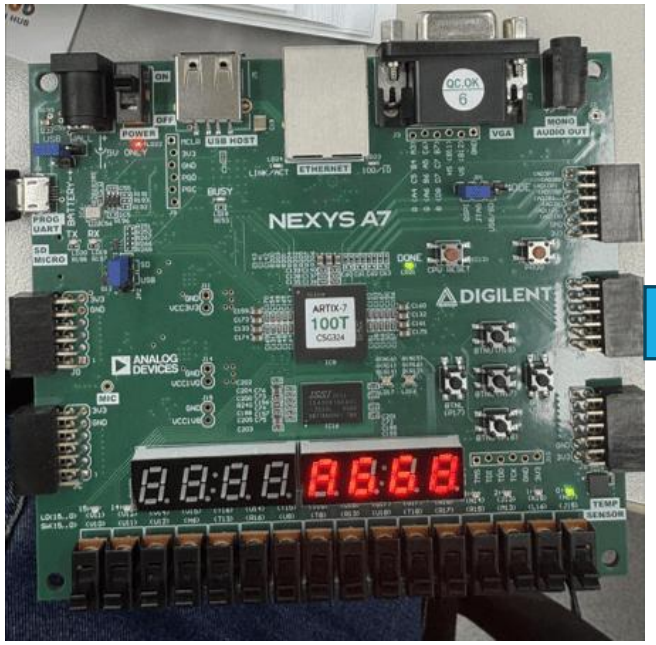


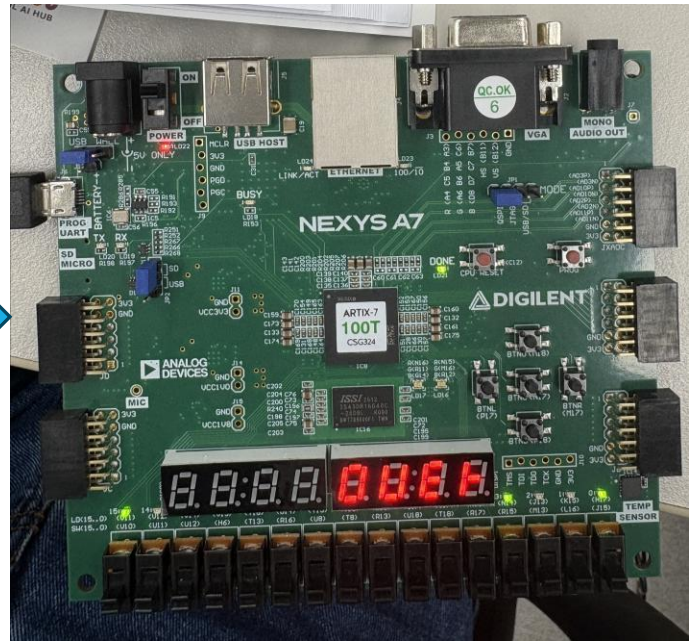
```
socmgr@soc1w18:sim_xrun

SEND: op(38) rs1(31) rs2(35) rd(32)
[time           4013545000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 8
[time :           4115535000] Operation select
[time           4202575000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           4304565000] rs1 select
[time           4391605000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 5
[time :           4493595000] rs2 select
[time           4580635000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           4682625000] rd select
[time :           4782865000] output result : 4'he [Passed]

SEND: op(39) rs1(31) rs2(35) rd(32)
[time           4970145000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 9
[time :           5072135000] Operation select
[time           5159175000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           5261165000] rs1 select
[time           5348205000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 5
[time :           5450195000] rs2 select
[time           5537235000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           5639225000] rd select
[time :           5739465000] output result : 4'ha [Passed]

SEND: op(45) rs1(31) rs2(32) rd(32)
[time           5926745000] [Host (tb) --> FPGA_SERIAL_RX] Sent char E
[time :           6028735000] Operation select
[time           6115775000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 1
[time :           6217765000] rs1 select
[time           6304805000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           6406795000] rs2 select
[time           6493835000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           6595825000] rd select
[time :           6696065000] output result : 4'h4 [Passed]

SEND: op(46) rs1(38) rs2(32) rd(32)
[time           6883345000] [Host (tb) --> FPGA_SERIAL_RX] Sent char F
[time :           6985335000] Operation select
[time           7072375000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 8
[time :           7174365000] rs1 select
[time           7261405000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           7363395000] rs2 select
[time           7450435000] [Host (tb) --> FPGA_SERIAL_RX] Sent char 2
[time :           7552425000] rd select
[time :           7652665000] output result : 4'h2 [Passed]
Simulation complete via $finish(1) at time 7753905 NS + 0
```

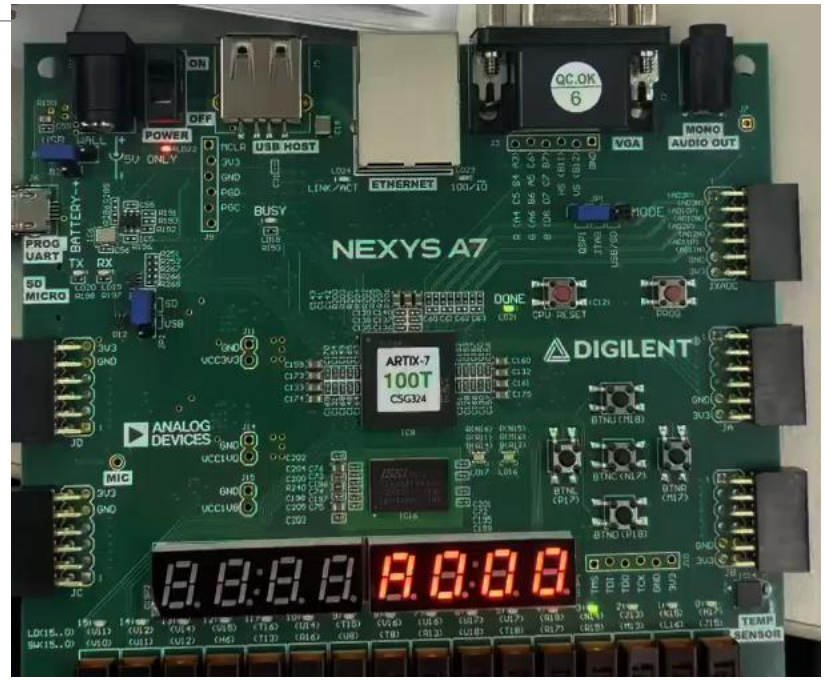


Opcode : Left shift
Rd1 : 1번 주소지 데이터
Rd2 : 사용자 입력값
Wr : 2번 주소지에 저장
0001 ----- > 0100

Final Project Milestone5

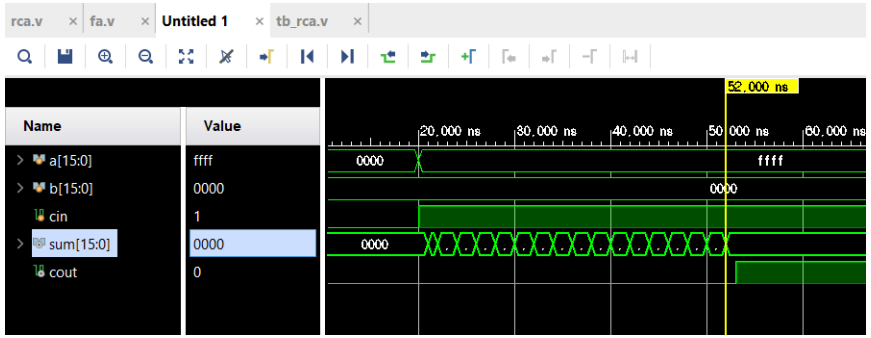


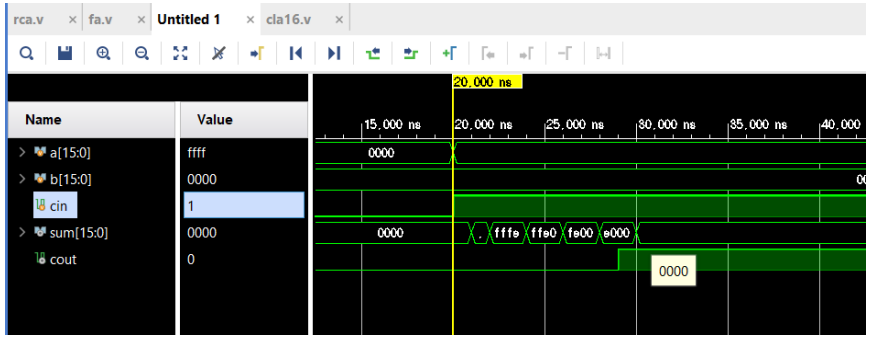




2의 보수 기반 4비트 signed 연산에서 6+6의 결과는 1100으로 나타나며, 최대 양수 0111을 넘어서 부호가 변하므로 오버플로우 발생

RCA – CLA 단일모듈 비교



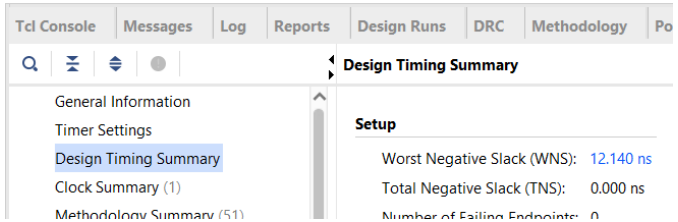


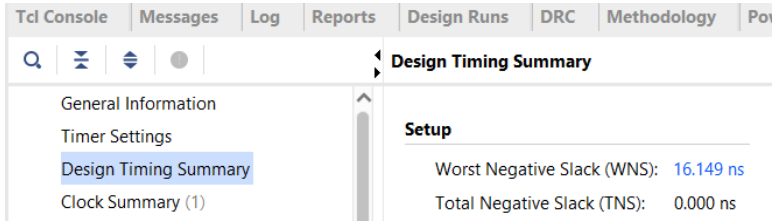
Cout 기준 76% 단축

Resource	Estimation	Available	Utilization...
LUT	16	41000	0.04
IO	50	300	16.67

Resource	Estimation	Available	Utilization...
LUT	29	41000	0.07
FF	50	82000	0.06
IO	52	300	17.33
BUFG	1	32	3.13

LUT 0.03%p 증가





타이밍 마진 33% 추가 확보

개인/프로젝트

PROJECT 01. 공정 미세화 한계 완화를 위한 PIM 개념의 Near-Memory 하드웨어 가속기 및 AXI 기반 SoC 통합 설계

수행기간

2025.12~2026.03

수행인원

3명 (스펙설정 및 모델링, RTL코딩 인터페이스 통합 및 FPGA 디버깅, Testbench 검증 및 보드 테스트)

사용기술

VIVADO(verilog), FPGA, ILA

수행내용

(1) Bottom to top으로 16bit CPU설계 및 UART로 들어오는 입력을 처리하는 제어 로직 구현

(2) 데이터값 및 연산모드 입력받기, 합산, 평균, 중앙값 연산하여 시스템 구축

(3) 커스텀한 CPU와 Xilinx사의 Bram Memory Controller IP와 연결. 버스 프로토콜로는 AXI4-Lite 사용 – 직접 만든 Master와 표준 Slave간의 연결 확인

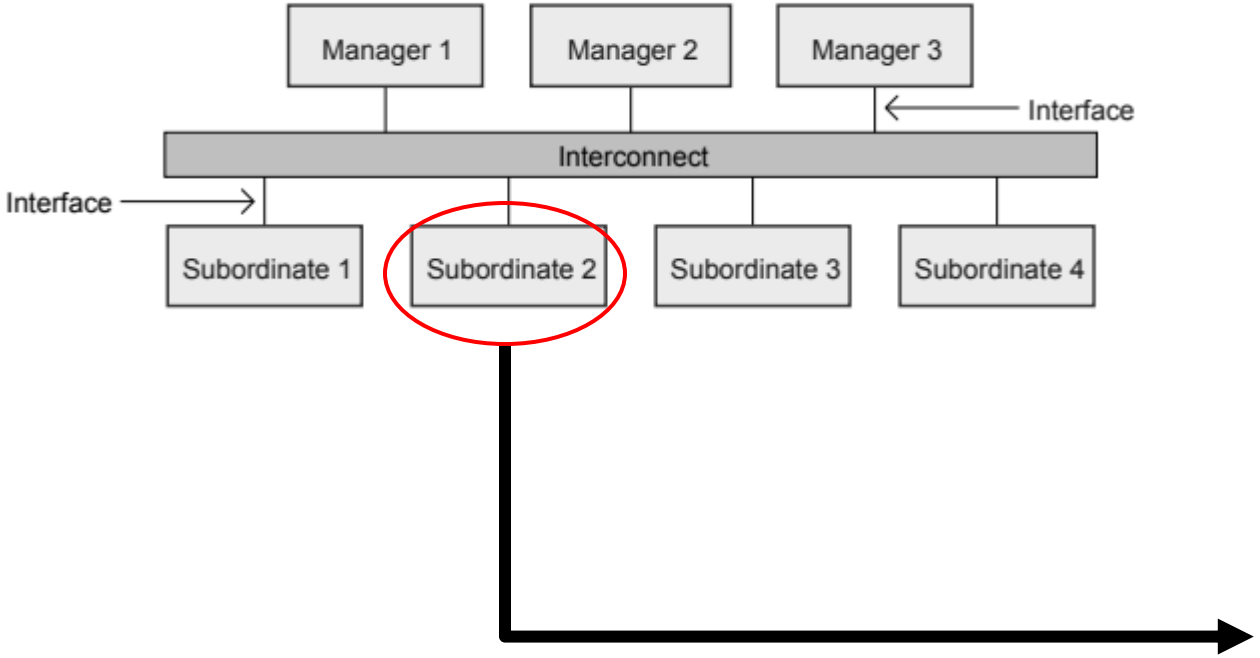
(4) PIM Slave 설계 – 단순 연산 시 CPU의 메모리 병목 해결, MAC 연산으로 AI 가속기로의 확장성 내제

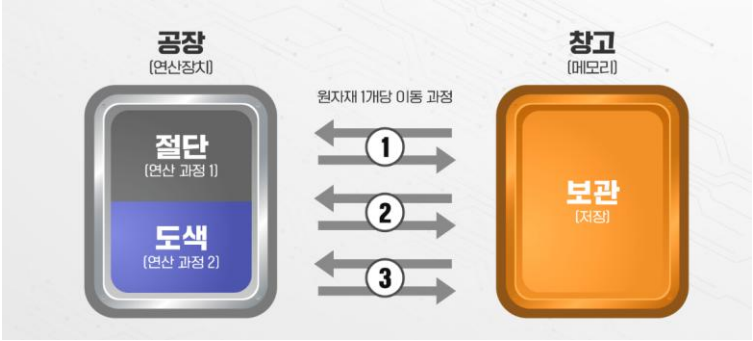
1. 서론 및 스펙 설정

미세 공정으로 갈수록 메모리와 연산기 사이의 물리적 거리에 따른 RC Delay와 Dynamic Power 소모가 극심해짐. 따라서 데이터 이동 자체를 최소화하는 AXI 기반 SoC 환경 구축 및, 메모리 병목을 제거하는 PIM 개념의 하드웨어 가속기 설계를 시도함

분류	스펙	비고
Core Architecture	16-bit Multi-Cycle CPU	커스텀 설계
Instruction Set	16-bit ISA	산술/논리/메모리/제어 명령어 자체 정의
System Bus	AMBA AXI4-Lite	32-bit Address / Data 패스
Memory System	Xilinx Standard BRAM Controller	표준 메모리 접근용 (MMIO 방식)
PIM controller	Custom RTL FSM Controller	커스텀 설계
Accelerator	Near-Memory MAC 가속기	PIM 개념 도입, DSP 할당
I/O Interface	UART (Rx/Tx), 7-Segment, Switch, LED	PC-FPGA 간 직렬 통신 및 상태 모니터링
Operating Freq	50MHz	FPGA보드 기본 클럭 주파수

2. 관련 연구



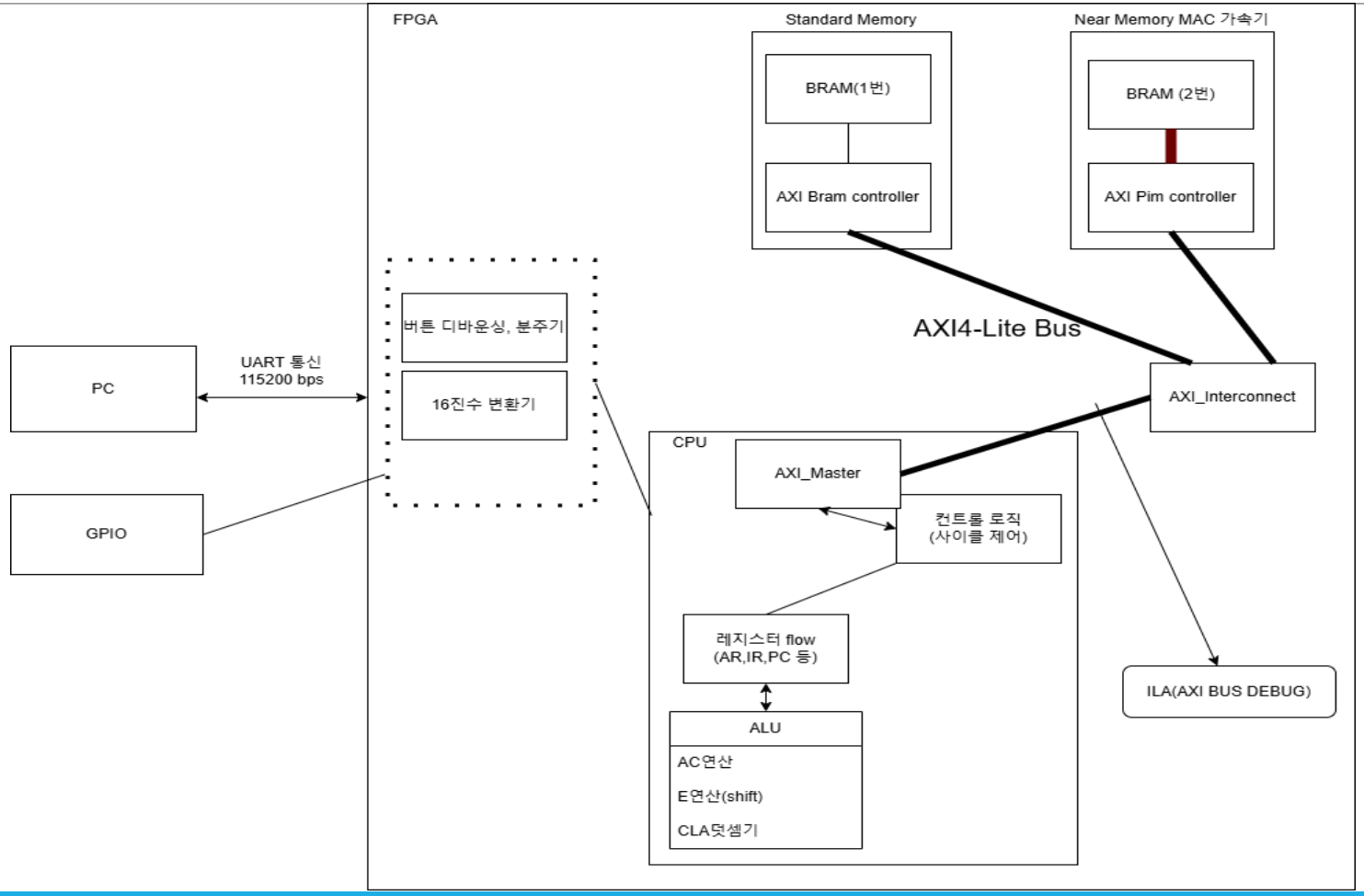


기존 폰노이만 구조

PIM 구조

출처 : SK HYNIX NEWSROOM

3. 블록 다이어그램


FPGA
Standard Memory
BRAM(1번)
AXI Bram controller
Near Memory MAC 가속기
BRAM (2번)
AXI Pim controller
AXI4-Lite Bus
AXI_Interconnect
버튼 디바운싱, 분주기
16진수 변환기
PC
UART 통신
115200 bps
GPIO
CPU
AXI_Master
컨트롤 로직
(사이클 제어)
레지스터 flow
(AR,IR,PC 등)
ALU
AC연산
E연산(shift)
CLA덧셈기
ILA(AXI BUS DEBUG)

4. 기능 정리

1번 모드
- 어셈블리 합산

2번 모드
- 어셈블리 평균

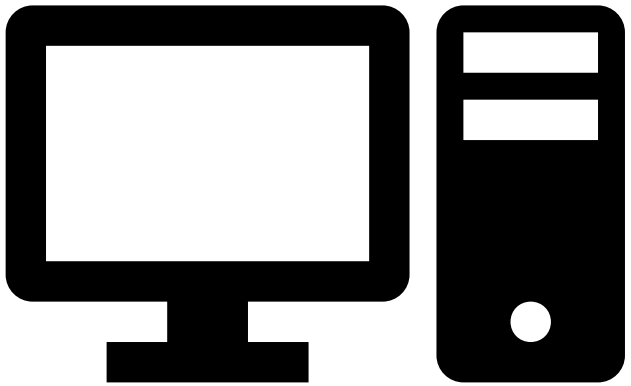
3번 모드
- 어셈블리 중앙값

4번 모드
- Pim 합산

5번 모드
- Pim 평균

6번 모드
- Pim MAC 연산

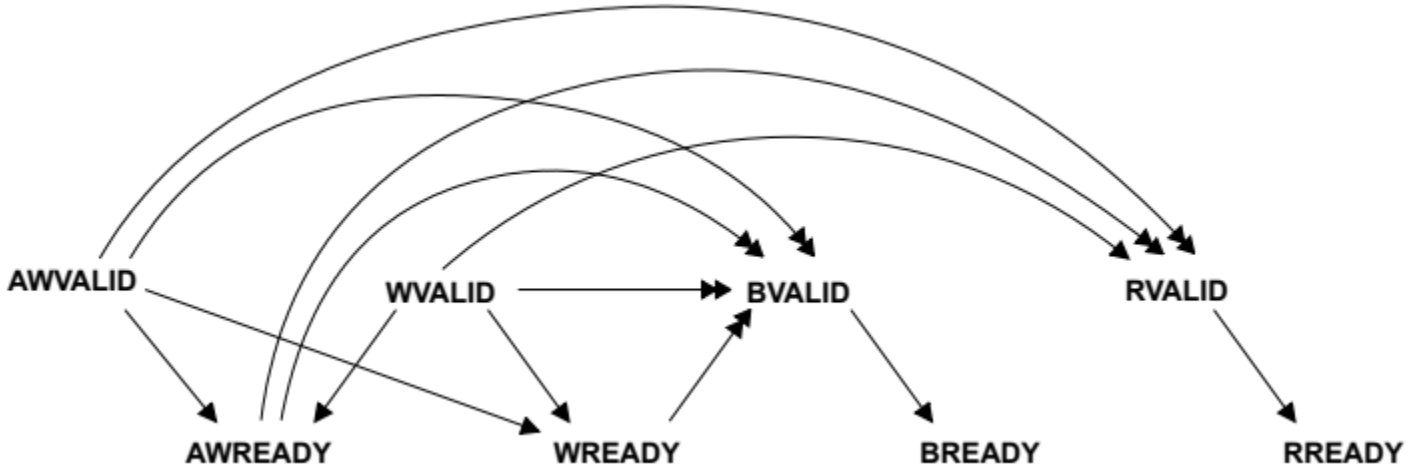
PC로부터 UART 통신으로 모드 및 데이터 입력



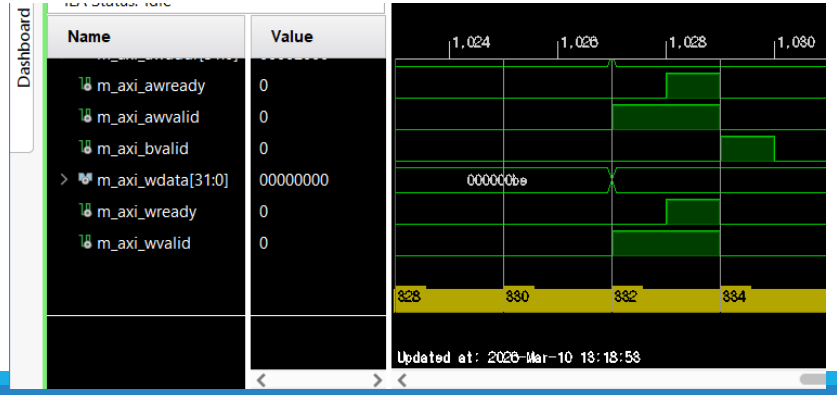
5. 핵심코드

: 서로 얽히지 않는 자유로운 관계

: 먼저 완료되어야 함 (데드락 방지)

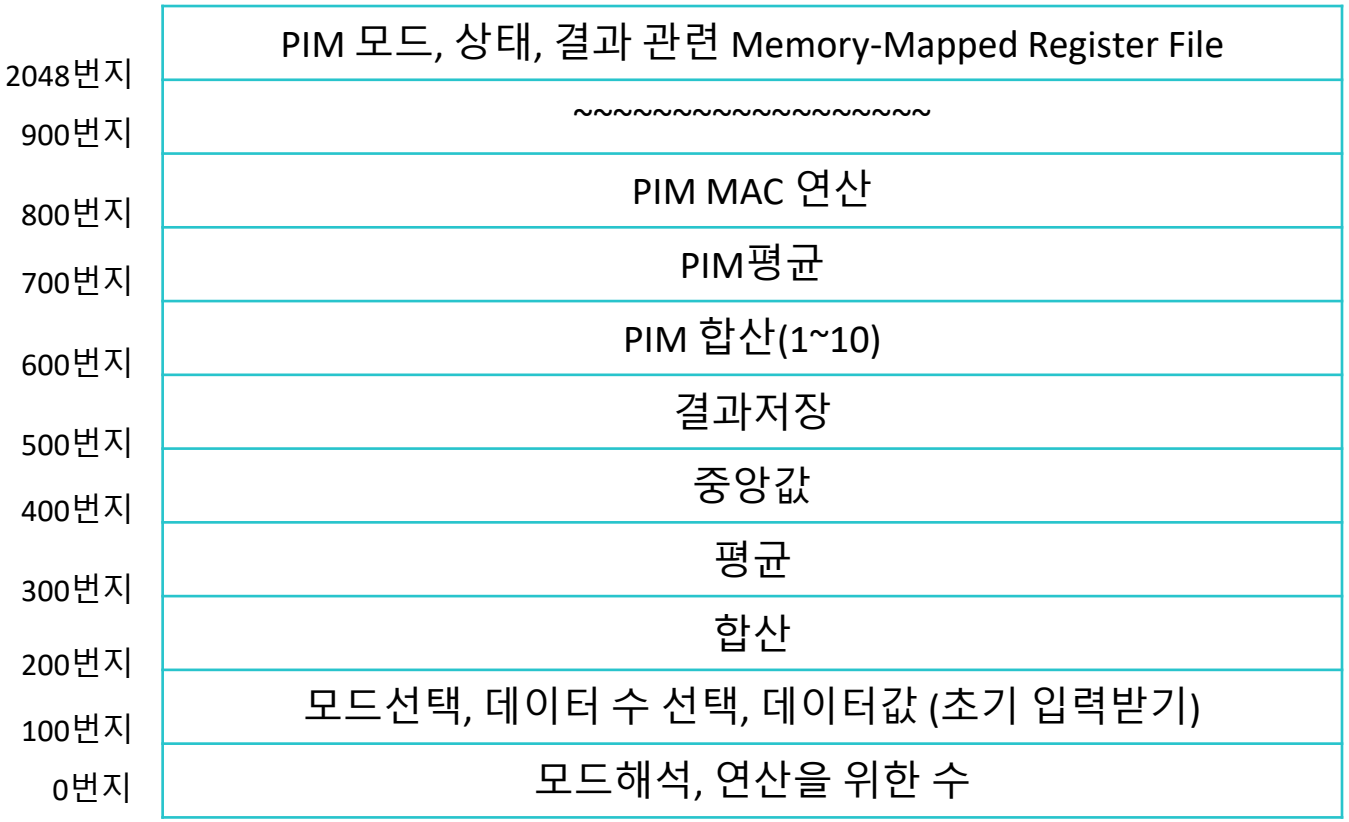



```
53          if (s_axi_awvalid && s_axi_wvalid && !s_axi_awready && !s_axi_wready) begin
54              s_axi_awready <= 1'b1;
55              s_axi_wready  <= 1'b1;
56              write_accept  <= 1'b1;
57
58              case (s_axi_awaddr[7:0])
59                  8'h00: slv_reg0_ctrl <= s_axi_wdata;
60              endcase
61          end
62          else begin
63              s_axi_awready <= 1'b0;
64              s_axi_wready  <= 1'b0;
65              write_accept <= 1'b0;
66          end
67
68          if (write_accept && !s_axi_bvalid) begin
69              s_axi_bvalid <= 1'b1;
70          end
71          else if (s_axi_bready && s_axi_bvalid) begin
72              s_axi_bvalid <= 1'b0;
73          end
```

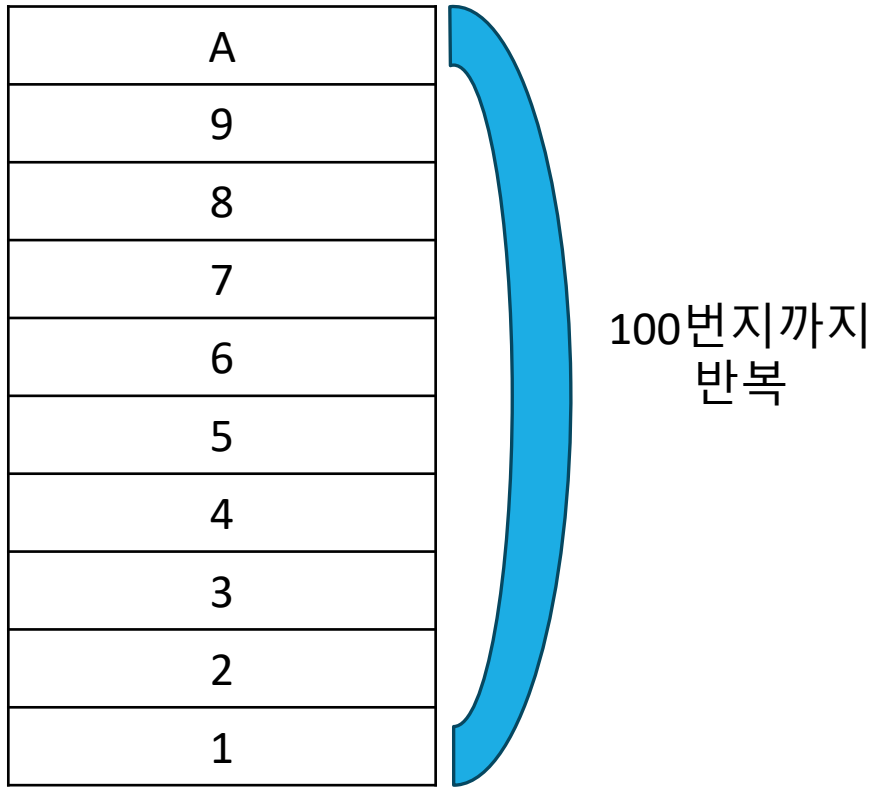


BVALID=1 조건 : AWVALID, WVALID, AWREADY, WREADY 모두 1인 상태

6. 메모리맵

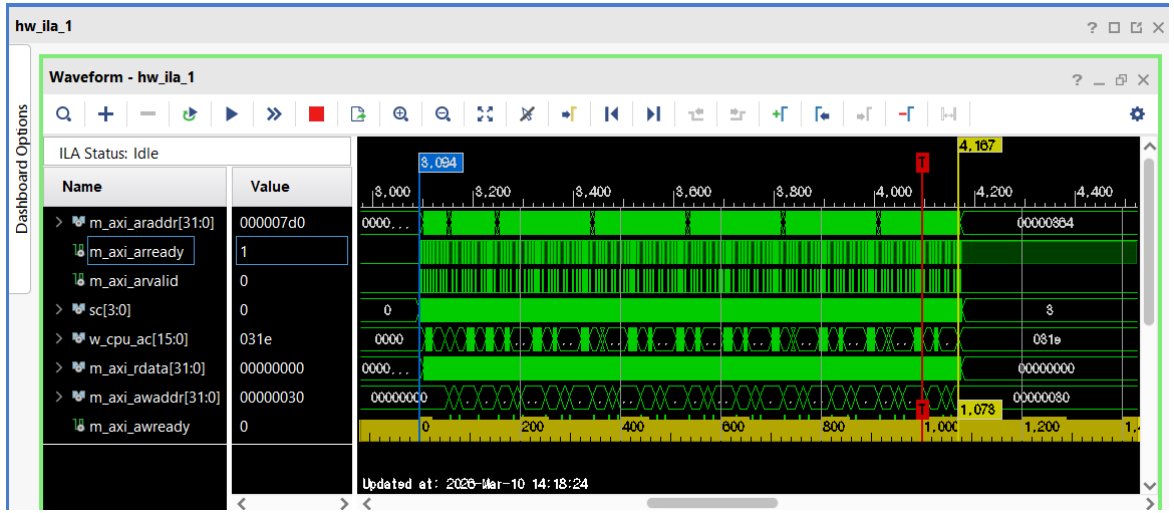


메인 메모리

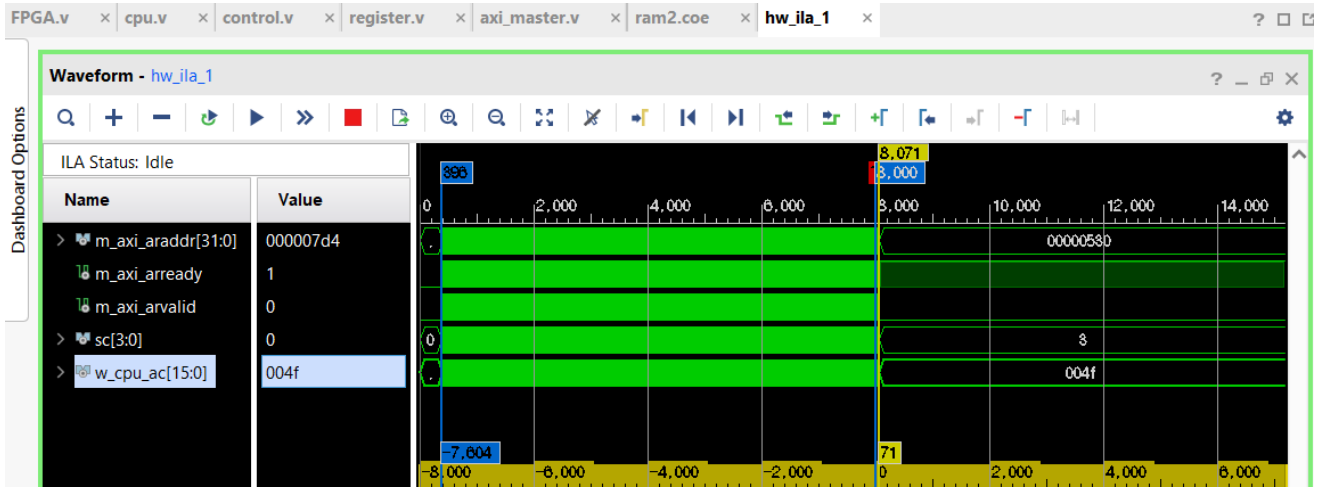


PIM 연동형 온칩 메모리

7. 결과(ILA 파형)

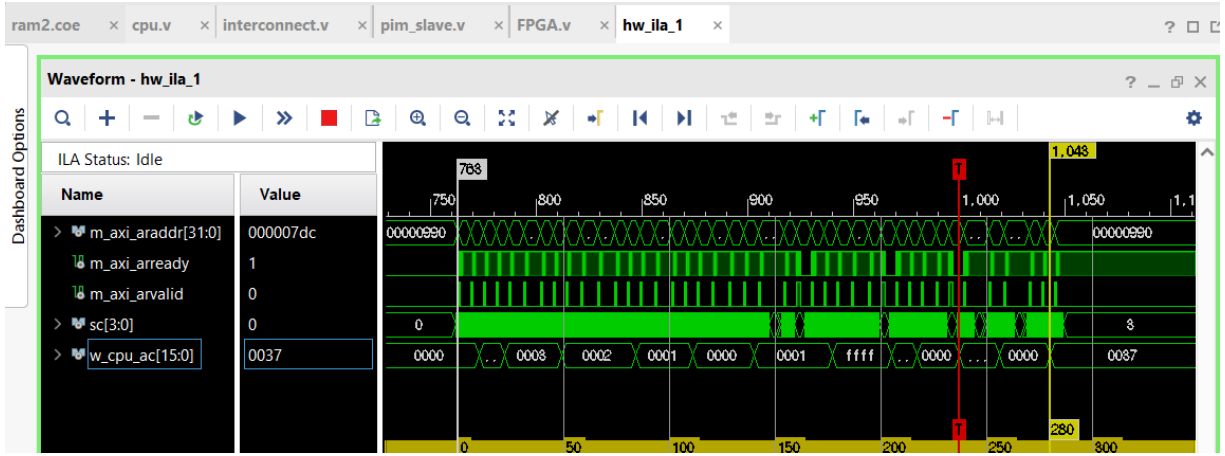


어셈블리 합산 1073 사이클 소요

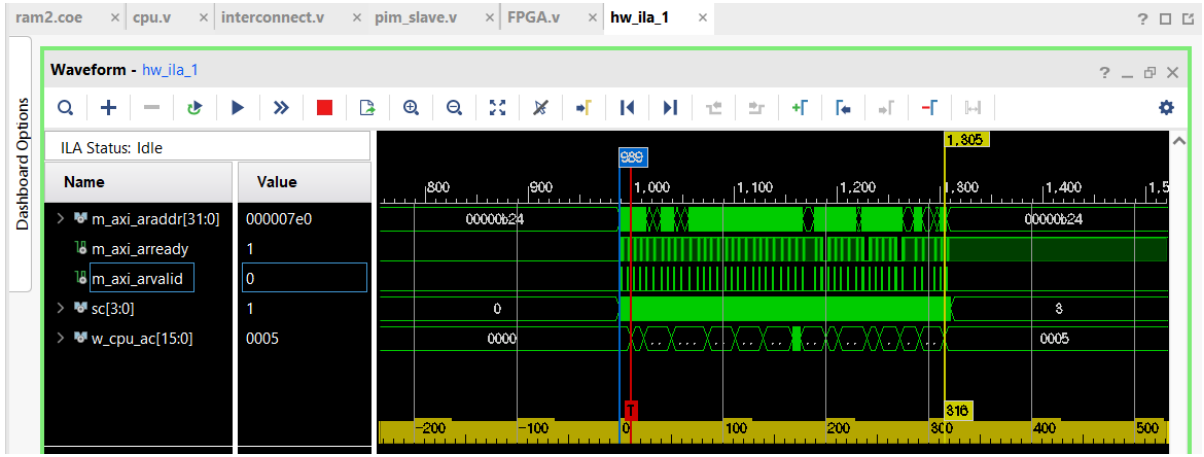


어셈블리 평균 7673 사이클 소요

7. 결과(ILA 파형)

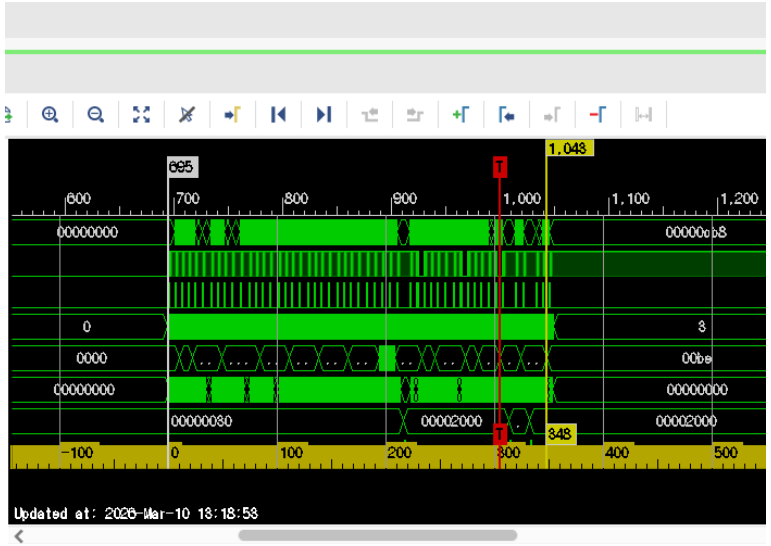


PIM 합산 280 사이클 소요

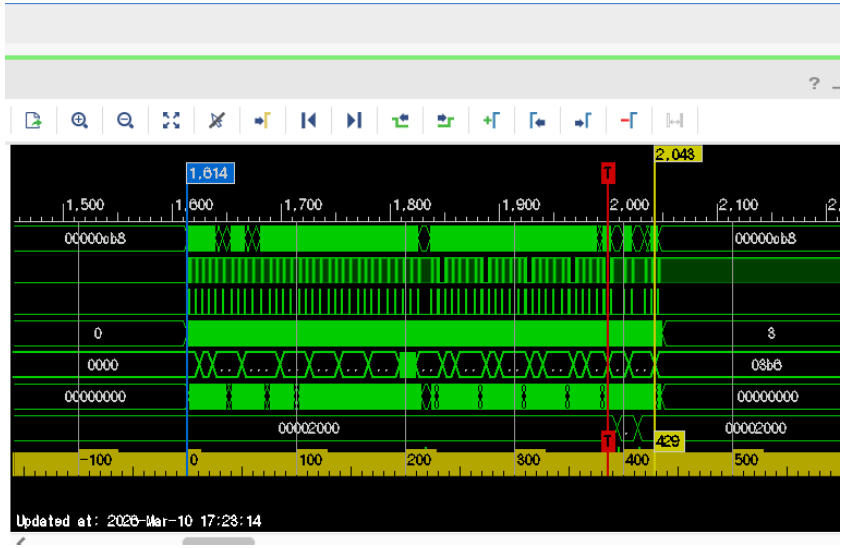


PIM 평균 316 사이클 소요

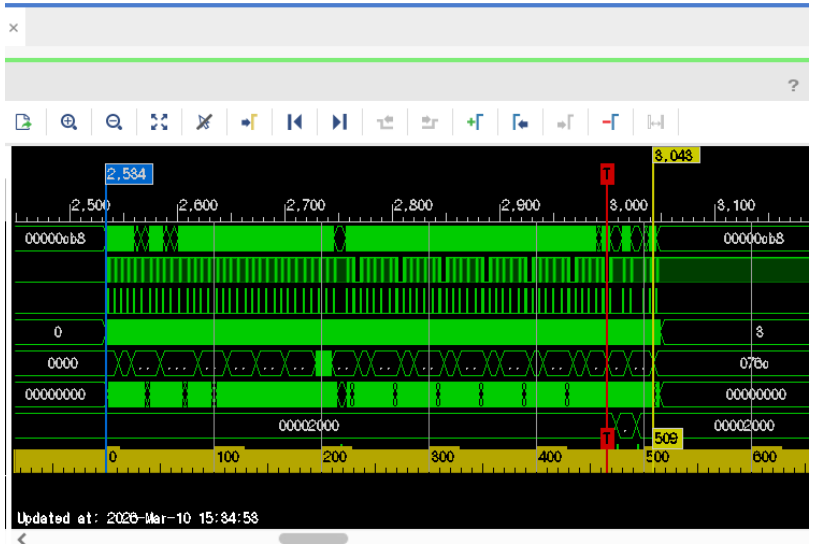
7. 결과(ILA 파형 – MAC연산)



10개 데이터
348사이클 소요

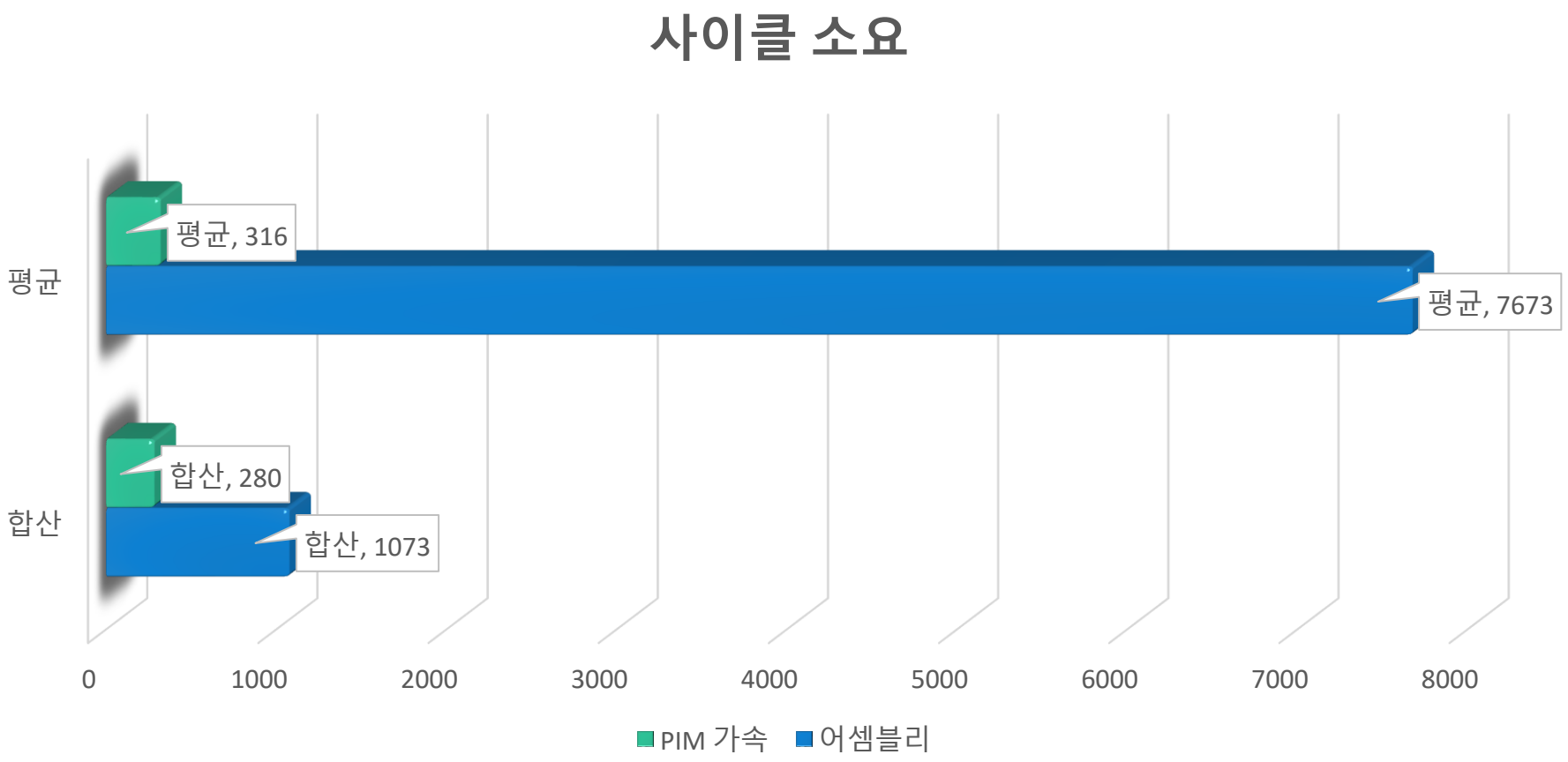


50개 데이터
429사이클 소요

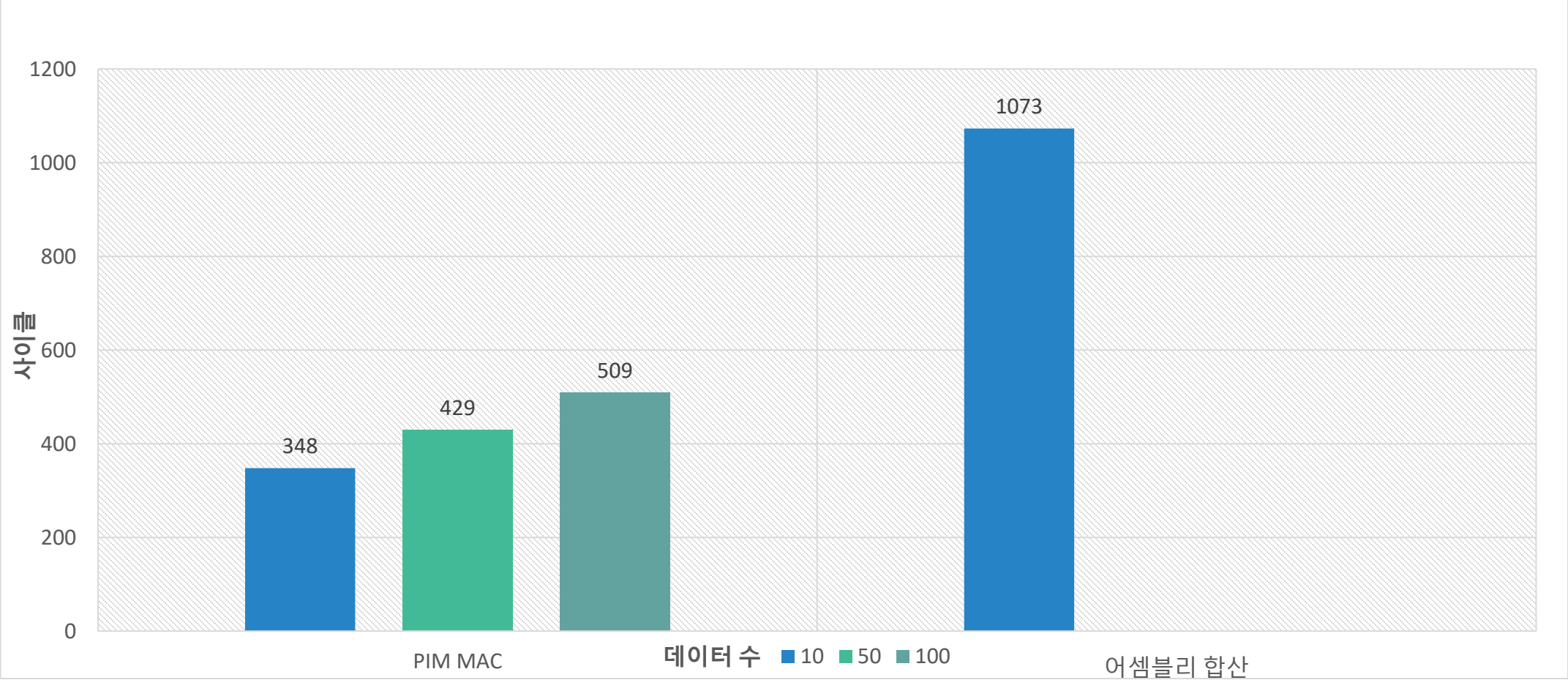


100개 데이터
509사이클 소요

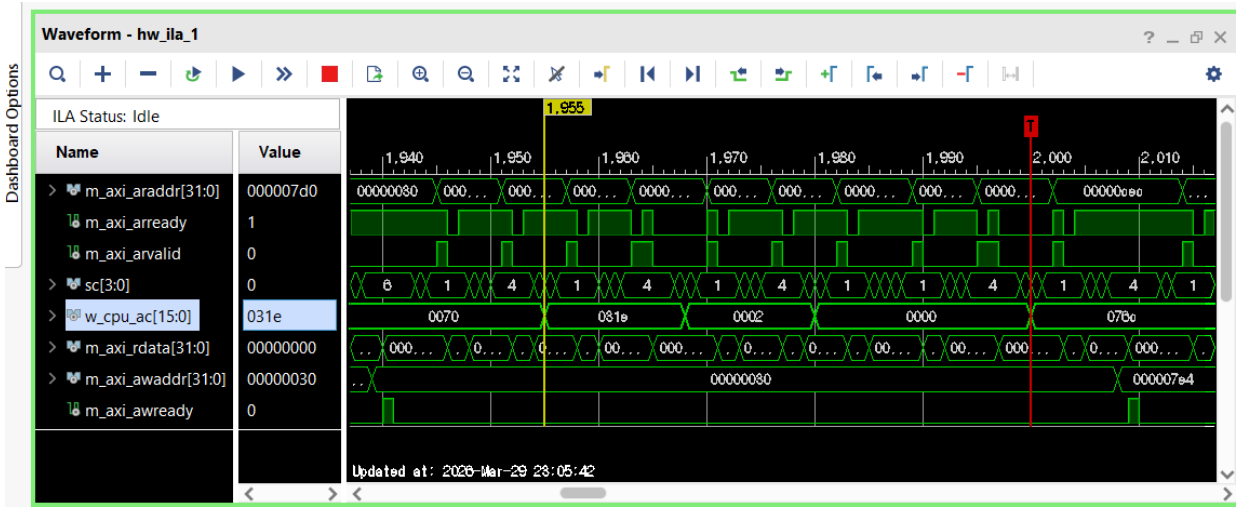
8. 결론 (사이클 비교)

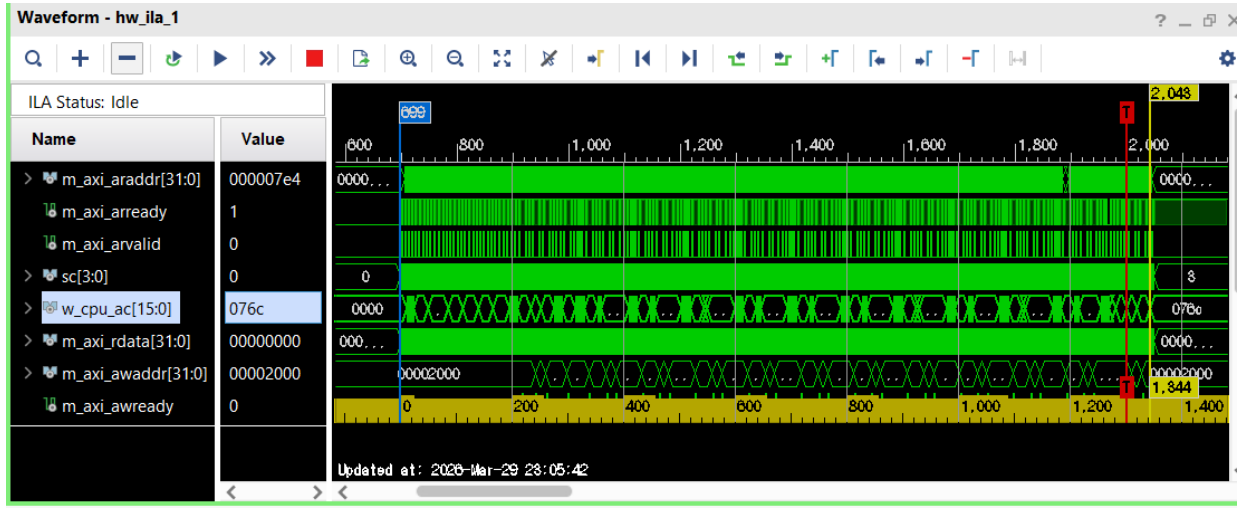

사이클 소요
평균
합산
평균, 316
평균, 7673
합산, 280
합산, 1073
0
1000
2000
3000
4000
5000
6000
7000
8000
PIM 가속
어셈블리

8. 결론 (기존 메모리 연산 대비 MAC 연산 사이클 비교)



8. 결론(CPU-PIM 비동기 병렬 처리 구현)





- 비동기 연산 오프로딩 및 병렬 처리 구조 설계

  CPU가 PIM에 MAC 연산을 지시한 이후, 기존의 폴링 방식으로 대기하지 않고, 해당 대기 구간 동안 독자적인 메모리 합산 연산을 수행하도록 재설계함. 이를 통해 단일 실행 흐름 내에서 두 개의 연산을 병렬적으로 수행하는 구조를 구현함.

- Throughput 향상 검증

  PIM 연산과 CPU 연산의 동시 수행을 통해 기존 순차 처리(1,582 Cycle) 대비 전체 실행 시간을 1,344 Cycle로 단축하여 약 15%의 성능 향상을 달성함.

8. 결론 (PIM - MAC)

- MAC 연산의 특징 : AI 알고리즘의 핵심은 두 벡터의 내적과 합(MAC)을 무한히 반복하는 것이다.

- 기존 폰노이만 구조의 한계 : 기존 CPU는 연산을 위해 메인 버스(AXI)를 통해 끊임없이 데이터를 병목 구간으로 끌어와야 하는 Memory Wall 현상에 직면해 있다.

- 해결책 (PIM 기반 가속기) : 시스템 버스의 개입을 최소화하고, 메모리에서 직접 연산을 초고속으로 처리하는 Near-Memory 하드웨어 가속기를 설계

- 공정/설계 친화적 장점:

  1) 상위 메탈 레이어 사용을 줄여 BEOL 라우팅 혼잡도 완화 및 국부적 발열 분산 (DFM 기여)
  2) 복잡한 버스 인터커넥트와 대형 캐시를 배제하여 다이 면적 효율성 극대화
  3) 자율주행/엣지 AI 환경 등 실시간 처리가 생명인 도메인에서 확고한 처리 시간 보장

8. 결론 (MAC 연산 Throughput)

$$\text{Throuput} = \frac{\text{처리 데이터 수}}{\text{총 } CYCLE}$$

10	• 0.0287
50	• 0.1166
100	• 0.1965

ILA 실측 결과, 입력 데이터 크기가 증가할수록 초기 latency의 영향이 감소하며, 추가 데이터 처리에 필요한 cycle은 약 2 Cycle/Data로 관측되었다.

Throuput의 증가를 확인하며 이는 본 MAC 구조가 데이터 크기가 증가할수록 안정적인 지속 처리율에 수렴함을 보여준다.

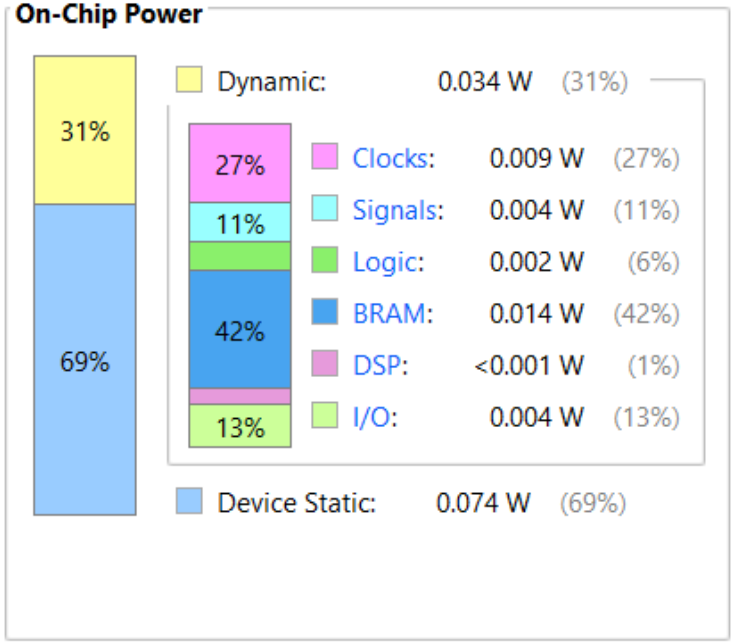
8. 결론(공정 미세화 한계 완화) - 전력소모 및 에너지)

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.108 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.3°C**
Thermal Margin:	59.7°C (21.2 W)
Effective ϑJA:	2.8°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity



총 소모 에너지 = 전력 × 소요시간

- 버스 오버헤드를 제거하고 4 Cycle/MAC 의 성능 달성
- 동일한 데이터량을 처리하는 소요시간 자체를 단축시켜 시스템 전체의 총 소모 에너지를 근본적으로 절감

8. 결론 (공정 미세화 한계 완화) – 배선 한계(RC)

Tcl Console | Messages | Log | Reports | Design Runs | DRC | Methodology | Power | Timing

Design Timing Summary
Clock Summary (2)
Methodology Summary (322)
Check Timing (500)
Intra-Clock Paths
clk
Setup 2.989 ns (10)
Hold 0.037 ns (10)

Intra-Clock Paths - clk - Setup

Name	Slack ^1	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	2.989	23	42	u_cpu/u_pim_sla...c_sum_reg[19]/C	u_cpu/u_pim_slav..._result_reg[8]/D	16.966	7.674	9.292	20.0
Path 2	3.016	23	42	u_cpu/u_pim_sla...c_sum_reg[19]/C	u_cpu/u_pim_slav...esult_reg[14]/D	16.935	7.674	9.261	20.0
Path 3	3.060	23	42	u_cpu/u_pim_sla...c_sum_reg[19]/C	u_cpu/u_pim_slav...esult_reg[16]/D	16.894	7.674	9.220	20.0
Path 4	3.066	23	42	u_cpu/u_pim_sla...c_sum_reg[19]/C	u_cpu/u_pim_slav..._result_reg[2]/D	16.887	7.674	9.213	20.0
Path 5	3.106	23	42	u_cpu/u_pim_sla...c_sum_reg[19]/C	u_cpu/u_pim_slav..._result_reg[6]/D	16.895	7.674	9.221	20.0
Path 6	3.107	23	42	u_cpu/u_pim_sla...c_sum_reg[19]/C	u_cpu/u_pim_slav...esult_reg[11]/D	16.845	7.674	9.171	20.0
Path 7	3.126	23	42	u_cpu/u_pim_sla...c_sum_reg[19]/C	u_cpu/u_pim_slav..._result_reg[7]/D	16.829	7.674	9.155	20.0

- Critical path 확인 결과, 전체 딜레이 대비 Net Delay가 약 54.8%를 차지
- 근본적인 원인을 줄이기 위해 PIM 도입으로 와이어 이동 자체 줄임
- 실제 공정 시 금속 배선 저항 관리가 성능의 핵심이 됨

8. 결론 (공정 미세화 한계 완화) – 타이밍 마진

Design Runs | DRC | Methodology | Pow

Design Timing Summary

Setup

Worst Negative Slack (WNS):	2.989 ns
Total Negative Slack (TNS):	0.000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	8955

All user specified timing constraints are met.

50MHz

Design Runs | DRC | Methodology | Pow

Design Timing Summary

Setup

Worst Negative Slack (WNS):	0.170 ns
Total Negative Slack (TNS):	0.000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	8955

All user specified timing constraints are met.

58.8MHz

Design Runs | DRC | Methodology | Po

Design Timing Summary

Setup

Worst Negative Slack (WNS):	-0.440 ns
Total Negative Slack (TNS):	-7.471 ns
Number of Failing Endpoints:	29
Total Number of Endpoints:	8955

Timing constraints are not met.

60MHz 초과

- PIM-SoC 아키텍처는 50MHz로 동작을 검증, Timing Summary 분석 결과 WNS 2.989ns를 확보
  - 이론적 최대 약 58.8MHz까지 오버클럭이 가능한 안정적인 설계
- 실리콘 양산 시 발생하는 PVT Variation 환경 속에서 타이밍 마진을 확보를 통해 견고한 물리적 설계 입증

8. 결론 (자원사용량)

	전체	사용량
LUT	20800	3373(16.22%)
LUTRAM	9600	285(2.97%)
FF	41600	3689(8.87%)
BRAM	50	50(100%)
IO	250	21(8.40%)
BUFG	32	2(6.25%)
DSP	90	3(3.33%)